Christopher Engert

Dokumentation & Reflexion Portfolio 3

Programmieren 2 – Bachelor Software Engineering Semester 2

Allgemeines Datenmodell:

Über alle drei Stufen hinweg verwende ich für die Repräsentation der extrahierten Log-Daten eine data class. Ich finde dieses Feature von Kotlin genial, weil es sich anfühlt, als würde man einen komplett eigenen, maßgeschneiderten Datentyp für das spezifische Problem erschaffen. Man bekommt Immutability sowie nützliche Funktionen wie toString() oder equals() geschenkt, ohne Boilerplate-Code schreiben zu müssen.

Stufe 1:

In der ersten Stufe lag der Fokus darauf, eine grundlegende funktionale Pipeline aufzubauen um Log-Zeilen zu filtern. Das Parsing der Zeichenkette passierte hier noch direkt innerhalb von .map.

Wenn ich mir den Code im Nachhinein ansehe, finde ich den Namen der Variable worte etwas missverständlich. Gemeint sind hier nicht die einzelnen Worte, sondern die Teile oder Fragmente der jeweiligen Log-Zeile.

Stufe 2:

In Stufe 2 wurde der Code modularisiert: Das Parsing wurde in eine eigene Funktion parseZeile ausgelagert, um Code-Duplizierung zu vermeiden. Zudem kam die Anforderung hinzu, eine Log-Statistik zu erstellen, die nach Häufigkeit absteigend sortiert ist.

In dieser Stufe arbeite ich ausschließlich mit normalen Kotlin-Standardlisten. Da Operationen wie .groupBy und .sortedByDescending in der Aggregation ohnehin die Gesamtheit aller Daten im Speicher sehen müssen, um korrekt zu sortieren, ist die Verarbeitung als Liste hier sinnvoll.

Stufe 3:

Die finale Stufe brachte die größte architektonische Herausforderung: Das System sollte absolut robust gegen fehleranfällige Log-Formate (z.B. Fehlende Doppelpunkte oder leere Nachrichten) werden. Da Exceptions in der Hauptlogik verboten waren, musste ein rein funktionales Error-Handling her. Gleichzeitig sollten fortgeschrittene Pipeline-Konzepte (Sequence, mapNotNull, flatMap) gezeigt werden.

KI-Support und reflektierende Worte:

Die finale Implementierung der Stufe 3 habe ich in intensiver Zusammenarbeit mit einer KI erarbeitet. Dabei ging es mir nicht darum, fertigen Code zu kopieren, sondern im Dialog die elegantesten funktionalen Entwurfsmuster zu verstehen.

In meinem ersten Entwurf für Stufe 3 dachte ich, ich müsste die Vorgabe des Aufgabenblattes stur erfüllen und mapNotNull und flatMap direkt hintereinander in einer einzigen Funktion erzwingen. Im KI-gestützten Code-Review haben wir das schnell als schlechten Stil entlarvt: Man packt Daten erst mittels mapNotNull aus, nur um sie direkt wieder in eine Liste zu stecken, damit flatMap sie wieder flachklopfen kann.

Die bessere Lösung war daher, die beiden Konzepte bewusst auf zwei Funktionen aufzuteilen. mapNotNull steuert die Fehlerfilterung in verarbeiteLogs unter Verwendung von performanten Sequenzen. flatMap wird stattdessen in aggregiereLogs genutzt. Beide Funktionen bleiben dadurch sauber und tun nichts Redundantes.

Der größte Lerneffekt aus den KI-Sessions war für mich ein Kotlin-Feature, welches ich vorher nicht auf dem Schirm hatte: Die Kombination aus dem Safe-Call-Operator (?.) und der Scope-Funktion let, die innerhalb des flatMap der Aggregations-Pipeline verwendet wird.

result.getOrNull() extrahiert den Wert und liefert im Fehlerfall null. Das ?.let wirkt wie eine Schranke, die sich nur öffnet, wenn der Wert ungleich null ist. Innerhalb von let repräsentiert it den sicheren Logeintrag. Dieser wird dann in eine Liste mit genau einem Element verpackt (listOf(it)). Ist der Wert null, wird der Block ignoriert und der Elvis-Operator (?:) gibt eine emptyList() zurück. flatMap nimmt diese Listen entgegen, flacht sie ab. Dadurch fließen gültige Einträge weiter, während die leeren Listen einfach aus dem Datenstrom verworfen werden.

Auch die Idee, .asSequence() an den Anfang der Pipeline zu setzen, stammt aus der KI-Beratung, um das speicherschonende Lazy-Evaluation-Prinzip zu nutzen. Statt bei jedem Schritt teure Zwischenlisten im RAM zu erstellen, fließen die Log-Einträge hier einzeln nacheinander durch die Kette. Das reduziert bei sehr großen Log-Dateien den Speicherverbrauch und macht die Verarbeitung effizienter. In aggregiereLogs wurde bewusst darauf verzichtet, da Sortier- und Gruppieroperationen ohnehin immer den gesamten Datensatz auf einmal im Speicher benötigen.

In diesem Portfolio habe ich trainiert, wie man den Kontrollfluss einer Anwendung komplett funktional hält, ohne den Datenstrom durch imperative, blockierende Kontrollstrukturen oder Exceptions zu unterbrechen.

Weitere KI-Unterstützung habe ich bei der Formulierung dieser Dokumentation in Anspruch genommen. Sie hat mir dabei geholfen, meine gedanklichen Ansätze und persönlichen Aha-Momente sauber zu strukturieren und präzise zusammenzufassen.